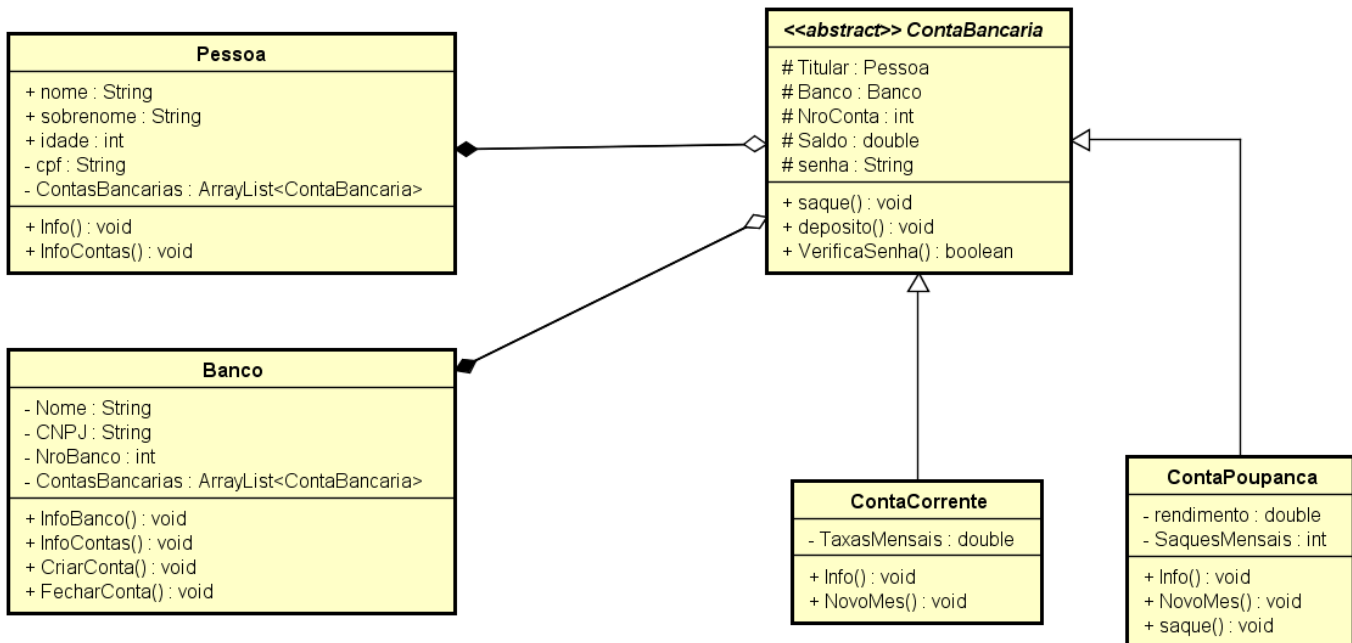


Programação Orientada a Objetos - Trabalho 3



Deve ser criada uma interface para usuário, onde seja possível cadastrar bancos, pessoas e contas, bem como obter informações dos objetos!

Detalhes de implementação:

Informações gerais:

- Devem ser implementadas formas de garantir que a entrada de dados não possa receber valores indevidos, por exemplo, um número de conta só pode aceitar valores inteiros positivos.
- Valores de saldo devem ser apresentados com somente 2 casas decimais e precedidos de R\$.
- Todas as classes devem possuir um método de classe (@classmethod) para construir os objetos através da entrada de dados por teclado.
- Atributos protegidos ou privados devem fazer uso de getters e setters (decoradores @property e @<property>.setter)

Persistência de dados:

- Os dados das contas, bancos e clientes devem ser armazenados em um arquivo.
 - A organização dos arquivos fica a critério do desenvolvedor
- Os dados devem ser carregados sempre que o programa iniciar.
 - Implementar método de classe (@classmethod) para construir os objetos através dos dados dos arquivos

Classe Conta Bancaria:

- **Método VerificaSenha:** Deve verificar se a senha de entrada corresponde a senha da conta e retorna um valor True ou False. Uma mensagem de aviso deve aparecer caso a senha esteja incorreta.
 - Implementar em dois formatos, um método VerificaSenha(int), que verifica diretamente a senha, e um VerificaSenha() que solicita a senha da conta pelo terminal.
- **Método saque:** Deve simular uma operação de saque, ou seja, o cliente entra com um valor e esse valor é retirado do saldo da conta. Deve utilizar o método VerificaSenha para permitir ou não o saque do valor.
 - Implementar em dois formatos, um método saque(float), que retira diretamente o valor, e um saque() que solicita o valor a ser sacado através do terminal
- **Método deposito:** Deve simular uma operação de depósito, ou seja, o cliente entra com um valor e esse valor é adicionado do saldo da conta.
 - Implementar em dois formatos, um método deposito(float), que deposita diretamente o valor, e um saque() que solicita o valor a ser depositado através de entrada via terminal.

Classe Conta Corrente:

- **Info():** Informa, nome do banco, número da conta, nome do cliente, saldo da conta e taxas mensais cobradas ao mês. (Pode ser feito através da sobre-escrita do método built-in `__str__`)
- **Método NovoMes():** simula a passagem de um mês na conta corrente, cobrando da conta o valor contido no atributo `TaxasMensais`.

Classe Conta Poupança:

- **Info():** Informa, nome do banco, número da conta, variação da poupança, nome do cliente, saldo da conta, rendimento mensal (%), saques restantes no mês. (Pode ser feito através da sobre-escrita do método built-in `__str__`)
- **Método NovoMes():** simula a passagem de um mês na conta corrente, aplicando o rendimento mensal no valor do saldo e retornando a quantidade de saques da conta para 3.
- **Método saque:** Deve simular uma operação de saque, ou seja, o cliente entra com um valor e esse valor é retirado do saldo da conta.
 - Para a conta poupança o cliente está limitado a 3 saques por mês, desta forma o cliente deve ser impedido de sacar caso já tenha feito 3 operações de saque.
 - Implementar em dois formatos, um método `deposito (float)`, que retira diretamente o valor, e um `deposito()` que solicita o valor a ser sacado através do terminal